



ARIA

AUTONOMOUS RESEARCH INTELLIGENCE AGENT

Portfolio Case Study

How I built an AI-powered research automation system using Claude, Notion, and structured prompt engineering

Claude AI
LLM Backend

Notion MCP
Knowledge Base

Web Search
Live Research

ReAct Loop
Agent Logic

Built with Claude · March 2026

Project Overview

ARIA (Autonomous Research Intelligence Agent) is an AI-powered research automation system I built entirely within Claude.ai, using structured prompt engineering, the Notion MCP connector, and Claude's built-in web search tool. The system discovers, analyses, and stores academic AI research automatically every day.

The Problem I Wanted to Solve

Keeping up with AI research is overwhelming. New papers appear daily across arXiv, NeurIPS, ICML, and dozens of other venues. Reading, summarising, and organising them manually is a full-time job.

The Solution

An autonomous agent that does the research work for me, searches the web for papers, analyses them using a structured reasoning framework, and stores everything in a searchable, organised Notion database, on a daily schedule.

What ARIA Does Daily

- Finds 3 high-impact AI papers
- Analyzes each with the ReAct framework
- Creates a Notion database entry per paper
- Writes a full analysis page inside each entry
- Checks for duplicates before storing
- Rates each paper 1–10 on importance
- Runs automatically at 10:00 AM IST

How I Built This With Claude

The entire system was built through a single Claude.ai conversation. No separate backend, no IDE, no deployment pipeline. Here's exactly what I did and how I instructed Claude at each step.

Step-by-Step Build Process

01

PROMPT ENGINEERING

Designed the ARIA system prompt using ReAct framework, AutoGPT-style loop, and strict output format templates. Fed structured instructions to Claude.

02

WEB SEARCH TOOL

Connected Claude's built-in web search tool (`web_search_20250305`) to enable real-time discovery from arXiv, Semantic Scholar, and Google Scholar.

03

NOTION MCP CONNECTOR

Enabled the Notion MCP connector in Claude's settings. This gave ARIA the ability to create databases, write page content, and manage entries, all from one API call.

04

DATABASE ARCHITECTURE

Designed a 14-property Notion database schema: AI Research Papers. Each paper = one row + a rich analysis page inside that row.

05

AGENT LOOP

Implemented AutoGPT-style 8-step loop: Plan → Discover → Filter → Analyse → Synthesise → Reflect → Store → Learn. Built deduplication into the storage step.

06

SCHEDULER

Built a browser-based IST scheduler using JavaScript intervals. At 10:00 AM IST daily, ARIA auto-triggers a full research cycle and populates the Notion database.

Prompt Engineering Breakdown

The core of ARIA is its system prompt, a ~800 word instruction set I carefully designed and iterated on. Here's how each layer of the prompt contributes to the system's behaviour.

PROMPT LAYER	WHAT IT DOES	TECHNIQUE
Identity Layer	Defined ARIA's name, role, and personality: analytical, mentor-like, curious. This shapes tone across all outputs.	✓
ReAct Framework	Required Claude to Reason → Act → Observe → Reflect before every research task. This prevents shallow outputs.	✓
AutoGPT Loop	8-step workflow: Plan, Discover, Filter, Analyse, Synthesis, Reflect, Store, Learn. Claude follows these steps autonomously.	✓
Output Template	Exact format spec for the daily report: Paper #1, Summary, Key Findings, Technical Contribution, Future Vision, Notion Status. This ensures parseable, consistent output.	✓
Notion Storage Rules	Instructed Claude to: check for duplicates first, fill all 14 properties, write page content using the Research Paper Analysis template, tag category, and set Status = New.	✓
DB Page Template	Defined a Notion page structure: Paper Summary, Key Findings, Technical Contribution, Why This Matters, Future Vision, Key Takeaways, Related Papers, and Personal Notes.	✓
Constraints Block	Hard rules: never hallucinate citations, never invent authors, state uncertainty explicitly, and prefer peer-reviewed sources.	✓
Self-Critique Mode	ARIA verifies its own output before finalising: papers real? citations accurate? summaries faithful? Insights logical?	✓

💡 Key Insight: Prompt as Architecture

Instead of writing code to control agent behaviour, I used the system prompt as the architecture itself.

The ReAct framework, the output format, the Notion storage rules, the self-critique mode are all defined in plain English.

Claude executes these instructions reliably because they are specific, structured, and unambiguous.

This is the core technique: treat the prompt like a program, not a suggestion.

What Information I Fed to Claude

Getting the right output required feeding Claude precise, structured information at each stage of the build. Here's what I provided and why.

1. The Initial Blueprint

I provided a detailed specification document covering:

- ARIA's identity, personality, and behavioural rules
- The ReAct reasoning framework (Reason → Act → Observe → Reflect)
- An AutoGPT-style 8-step task execution loop
- An exact output format template for the research report
- Notion storage rules, including the database schema and page template
- Self-critique mode requiring ARIA to verify its own outputs

2. The Notion Database Architecture

When I asked for a richer knowledge base, I provided Claude with:

- The database name: AI Research Papers
- All 13 property definitions with types (title, rich_text, number, select, multi_select, url, date)
- 10 Research Category options for the multi-select tag
- 5 Notion view definitions with their filter/sort rules
- The page content template: Research Paper Analysis with 8 sections
- Duplication rules: check title before creating, skip if exists

3. Iterative Refinements

The system evolved through several rounds of feedback:

- Added the Notion DB creation step with a one-click setup flow
- Added the daily scheduler with IST timezone (10:00 AM) and countdown timer
- Upgraded the database architecture to database-as-main-window with row → page navigation
- Added 5 filtered Notion views (Latest, Most Influential, AI Agents, Deep Dive Queue, High Impact)
- Added deduplication logic to the storage workflow
- Added the Importance Score property and Self-Critique mode

Connectors & Tools Used

Notion MCP Connector

The Notion MCP (Model Context Protocol) connector is enabled in Claude's settings. It gives Claude direct access to the Notion API, allowing ARIA to:

MCP Server Config

```
type: "url"
url: "https://mcp.notion.com/mcp"
name: "notion-mcp"
```

- Create new databases with custom schemas
- Create database entries (rows) with all properties filled
- Write structured page content inside each entry
- Search existing entries for deduplication
- Create filtered views within the database

No API keys were needed. The connector is authenticated through Claude's interface and the MCP server URL is passed directly in the API call.

Web Search Tool

Claude's built-in web search tool (type: web_search_20250305) enables real-time paper discovery. ARIA uses it to:

- Search arXiv for recent preprints
- Query Semantic Scholar for citation counts
- Find papers from Google Scholar
- Verify paper existence before citing

Notion Database Design

The database architecture was designed to be both a research log and a knowledge base. Each paper gets a row in the database table and a full analysis page inside that row — mimicking how Notion's own databases work.

Database Schema: AI Research Papers

PROPERTY	TYPE	PURPOSE
Title	title	Paper title: the primary key
Authors	rich_text	Author names
Year	number	Publication year for sorting
Publication	select	arXiv / NeurIPS / ICML / ACL / ICLR / CVPR
Paper Link	url	Direct link to the paper
Citation Count	number	Influence metric: used in Most Influential view
Research Category	multi_select	LLM Architecture, AI Agents, RAG, AI Safety, etc.
Key Idea	rich_text	One-sentence summary of the paper's core contribution
Significance	rich_text	Why this paper matters now
Future Impact	rich_text	Long-term implications for AI development
Date Added	date	Auto-set by ARIA: powers the Latest Research view
Status	select	New / Reviewed / Deep Dive
Importance Score	number	ARIA's rating 1–10: powers High Impact view

5 Smart Views

Each view is a different lens on the same database: no duplicate data, just filtered perspectives:

VIEW NAME	FILTER / SORT	USE CASE
 Latest Research	Sort by Date Added ↓	See the most recently discovered papers first
 Most Influential	Sort by Citation Count ↓	Surface highest-impact papers from history
 AI Agents Research	Filter: Category = AI Agents	Focused lens on autonomous agent papers only

VIEW NAME	FILTER / SORT	USE CASE
 Deep Dive Queue	Filter: Status = New	Your reading queue: all unreviewed papers
 High Impact Papers	Filter: Importance Score ≥ 8	ARIA's top-rated papers across all categories

 Page-Inside-Row Architecture

The most powerful design decision: each database row opens as a full Notion page. The page follows the Research Paper Analysis template with 8 structured sections. This means your database is a table of contents, and clicking any row gives you the full deep-dive. ARIA writes the entire page content automatically; you just read and annotate.

The Daily Research Scheduler

One of the most impressive features, ARIA runs automatically every morning at 10:00 AM IST (Asia/Kolkata) without any manual trigger.

How It Works

- A JavaScript setInterval runs every 30 seconds
- Each tick computes the current IST time (UTC + 5:30)
- When the clock hits 10:00 AM, the scheduler fires
- A deduplication flag prevents double-runs on the same day
- ARIA searches for 3 papers, analyses them, and stores them in Notion
- The run is logged in the Run History with a timestamp and status

The tab must remain open for scheduled runs. For production, this can be replaced with a server-side cron job, n8n workflow, or Zapier automation.

Scheduler Spec

Timezone: Asia/Kolkata (IST)
Schedule: Every day at 10:00 AM
Trigger: setInterval every 30s
Dedup: runs once per calendar day
Fallback: manual trigger button
History: timestamps + success/fail log
Status: live countdown timer

Key Learnings & Techniques

What Made This Work

- **Specificity in prompts:** Vague instructions produce vague results. Every rule in ARIA's system prompt is specific and unambiguous: exact format templates, explicit constraints, and named database fields.
- **Structured output templates:** Defining an exact output format (PAPER #1, SUMMARY, KEY FINDINGS, etc.) makes the response machine-parseable, not just human-readable.

- **The ReAct framework:** Requiring Claude to reason before acting prevents shallow, rushed outputs. The agent genuinely thinks through the research task.
- **MCP connectors as superpowers:** The Notion MCP connector turned ARIA from a chatbot into a genuine automation system. No code, no API keys — just enabling a connector.
- **Iterative refinement:** The final system emerged through 6+ rounds of feedback. Each iteration added a feature or improved existing behaviour.
- **Database-as-knowledge-base:** The key architectural insight: a database row that opens as a full analysis page is far more valuable than a flat text log.

Techniques I Used to Instruct Claude

- Fed structured specification documents as context before making requests
 - Used numbered and formatted lists to make instructions scannable and precise
 - Defined the exact output format templates Claude should follow
 - Specified constraints explicitly (never hallucinate, never duplicate, etc.)
 - Asked for features incrementally rather than all at once
 - Described the 'why' behind each requirement so Claude could make better decisions
 - Used connector names and MCP server URLs directly in my instructions
 - Tested each feature before adding the next one
-

System Architecture Summary

LAYER	COMPONENT	ROLE
Frontend	React (JSX) Artifact	Chat UI, Database view, Agent trace, Scheduler panel
Agent Brain	Claude claude-sonnet-4-20250514	Research, analysis, synthesis, self-critique
Web Research	web_search_20250305 tool	Real-time arXiv, Semantic Scholar discovery
Knowledge Base	Notion MCP Connector	Database creation, entry storage, page writing
Scheduler	JavaScript setInterval	Daily 10:00 AM IST automated trigger
Memory	Conversation history array	Multi-turn context for follow-up queries
Deduplication	Title matching in storage prompt	Prevents duplicate database entries

What This Demonstrates

This project shows that with the right prompting strategy and Claude's connector ecosystem, it's possible to build a genuinely useful autonomous system, without writing a single line of backend code.

The combination of:

- Structured system prompts (defining agent behaviour)
- MCP connectors (giving Claude real-world actions)
- Built-in tools (web search for live data)
- Iterative refinement (improving through conversation)

...produces a system that would normally require a backend server, a database, multiple API integrations, and weeks of engineering work.

The Core Thesis

Prompt engineering is software engineering, when done with the same rigour and structure.
Claude + connectors + the right instructions = a deployable autonomous agent.
This project was built entirely in conversation. No IDE. No deployment. Just prompts.

Built with Claude AI · March 2026